Lab Report: Lab 7 - Event-Driven Architecture (EDA) & Integration

Contents

---

1. Cover Page

- **Title:** Implementing Event-Driven Architecture and Asynchronous Integration for OmniDash
- **Course Info:** Software Architecture
- **Student Details:** Nguyễn Văn Trường, 23010371
- **Date:** April 20, 2026

---

2. Abstract/Summary

As the OmniDash platform scales, relying solely on synchronous HTTP communication introduces performance bottlenecks and tight coupling, particularly for non-critical, time-consuming tasks. This lab introduces the Event-Driven Architecture (EDA) pattern to achieve asynchronous decoupling. We successfully integrated RabbitMQ as a centralized Message Broker operating via Docker. Within this ecosystem, we engineered a Producer service (e.g., Task Service) responsible for publishing state-change events (e.g., TaskCreatedEvent) to a designated queue. Concurrently, a separate Consumer service (Notification Service) was implemented to listen to this queue, process the events asynchronously (e.g., sending email alerts), and acknowledge successful execution. The primary achievement of this lab is the empirical demonstration of extreme fault isolation: the Producer can successfully execute its core business logic and emit events even if the Consumer is temporarily offline, ensuring high system availability and zero data loss.

---

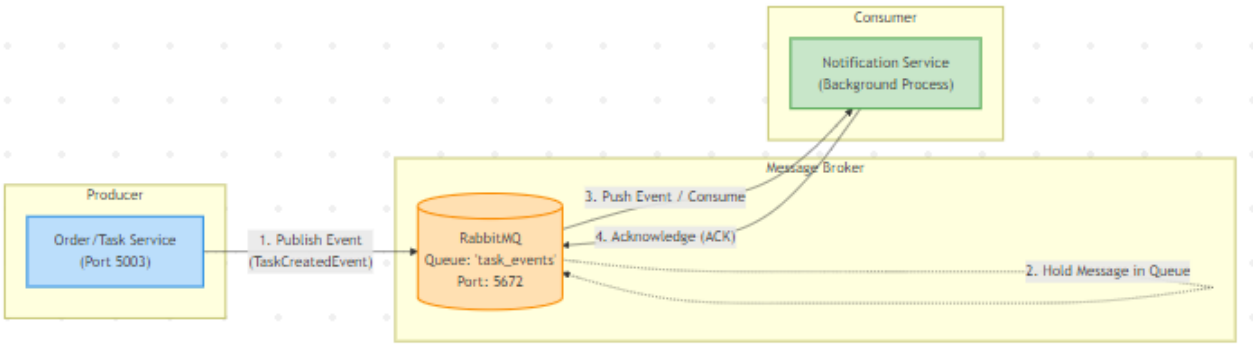
3. Lab Specific Section: Event-Driven Integration

This section documents the architectural strategy and logical implementation of asynchronous messaging within the OmniDash microservices ecosystem.

3.1 The Necessity of Asynchronous Decoupling

In the previous labs, microservices communicated synchronously via the API Gateway. While synchronous REST is excellent for immediate data retrieval, it is highly inefficient for background operations. For instance, when an OmniDash user creates a new highly-priority Task or Workspace, the system might need to send a confirmation email or a push notification. If the Task Service were to call the Notification Service synchronously, the user would be forced to wait for the email transmission to complete before the UI updates. Furthermore, if the Notification Service crashed, the entire task creation process would fail, violating the principle of fault isolation.

Event-Driven Architecture (EDA) resolves this by introducing a "Fire-and-Forget" mechanism using a Message Broker.



3.2 Setting Up the Message Broker (RabbitMQ)

The backbone of this architecture is the Message Broker. We utilized RabbitMQ, an industry-standard message-queueing software, deployed containerized via Docker to ensure environment consistency. RabbitMQ operates on the Advanced Message Queuing Protocol (AMQP). It acts as a reliable intermediary, holding messages in persistent queues until they are successfully consumed, thus providing a buffer that absorbs traffic spikes and prevents data loss during service downtimes.

3.3 The Producer Logic (Publishing Events)

The Producer represents the microservice where a significant state change occurs. In OmniDash, this could be the Task Service (or Order Service in e-commerce contexts).

When a user successfully creates a task, the Producer executes its core database transaction (saving the task to PostgreSQL). Immediately after the successful commit, the Producer constructs an Event Payload—a serialized JSON dictionary containing essential context (e.g., task_id, user_email, timestamp). The Producer establishes an AMQP connection to RabbitMQ and publishes this JSON payload to a specific routing queue (e.g., task_events). Crucially, once the message is dispatched to the broker, the Producer immediately closes the connection and returns an HTTP 201 Created response to the user. The Producer does not wait to see if the notification was actually sent.

4. Conclusion & Reflection

Lab 7 successfully transitioned OmniDash from a synchronous network into a highly resilient Event-

Driven Architecture. By integrating RabbitMQ, we effectively decoupled the core business logic (Task/Workspace creation) from secondary, time-consuming operations (Notifications).

The testing phase proved the immense value of this pattern: the Producer remains highly responsive and unaffected by the Consumer's operational status. This fault isolation ensures that a failure in the email system will never prevent a user from utilizing the core OmniDash platform. This asynchronous capability is a hallmark of enterprise-grade microservices and sets the stage for analyzing the overall system's scalability and availability in the final deployment lab.